SIMATS ENGINEERING
ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION **COURSE CODE: ITA0509**

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.* (BTL 6)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Annexure A
Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I **N.AKSHAYA(192425129)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Q. No	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Complete ness (20%)	Total Score (100%)
1						
2						
3						
4						
5						

Total Marks (Out of 100)		Total Marks (Out of 50)	

Annexure A
Pseudocode Writing Task (Algorithm Design)

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Face Detection and Recognition Pipeline

A system must process a live video stream to:

A. Capture frames from a camera
B. Detect faces in each frame
C. Recognize faces using a pre-trained dataset
D. Display bounding boxes and labels in real time

Constraints:

- Must operate continuously
- Should minimize processing delay
- Should handle multiple faces

Write detailed pseudocode for the complete pipeline using OpenCV concepts, including image acquisition, preprocessing, detection, recognition, and display. Ensure modular design and efficient looping structure.

Answer:

A. Problem Understanding:

- **Input:** Live camera video and pre-trained face dataset.
- **Output:** Detected faces with names and bounding boxes.
- **Constraints:** Continuous operation, low delay, and multiple faces.

B. Algorithm Design:

```
START
Load face detection and recognition models
Open camera
WHILE camera is active
   Capture frame
   Preprocess frame
   Detect faces
   FOR each face
      Recognize face
      Draw bounding box and name
   END FOR
   Display frame
   IF exit key is pressed
      BREAK
   END IF
END WHILE
```

Release camera
END

C. Use of Control Structures:

- WHILE continuously processes camera frames.
- FOR handles multiple faces.
- IF checks the exit condition.

D. Pseudocode Structure:

pseudocode follows a clear sequence:
camera input → preprocessing → face detection → face recognition → labeling → display.

Completeness:

The algorithm detects and recognizes multiple faces continuously and displays their labels and bounding boxes.

2. OCR Pipeline for Document Processing

A document processing system must:

A. Read input images containing printed text
B. Enhance image quality (noise removal, contrast improvement)
C. Segment text regions
D. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

Answer:

A. Problem Understanding:

- **Input:** Printed-text document images.
- **Output:** Recognized text.
- **Constraints:** Noisy/low-contrast images and batch processing.

B. Algorithm Design:

```
START
Load input document images
FOR each image
   Read image
   Convert to grayscale
   Remove noise
   Improve contrast
   Apply thresholding
   Segment text regions
   Extract text using OCR
   Display recognized text
END FOR
END
```

C. Use of Control Structures:

- FOR processes multiple document images.
- Processing steps are applied sequentially to each image.

D. Pseudocode Structure:
The pseudocode is organized into image reading
image reading → preprocessing → segmentation → OCR → text display.
Completeness:
The algorithm handles noisy images, improves image quality, segments text, and extracts recognized text.

3. Motion Detection and Tracking System
A surveillance system must:

A. Read video input
B. Detect motion between consecutive frames
C. Track moving objects
D. Highlight detected motion regions

Constraints:

- Should handle dynamic background
- Must operate in near real-time

Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.
Answer:
A. Problem Understanding:

- **Input:** Continuous surveillance video.
- **Output:** Detected and tracked moving objects.
- **Constraints:** Dynamic background and near-real-time operation.

B. Algorithm Design:

```
START
Open video
Read previous frame
WHILE video is available
   Read current frame
   Convert frames to grayscale
   Calculate frame difference
   Apply threshold and filtering
   Detect moving regions
   Track moving objects
   Draw bounding boxes
   Display frame
END WHILE
Release video
END
```

C. Use of Control Structures:

- WHILE continuously processes video frames.
- Filtering and detection conditions identify valid motion regions.

D. Pseudocode Structure:
The pseudocode follows
video input → frame comparison → filtering → motion detection → tracking → visualization.

Completeness:
The algorithm detects motion, filters unwanted regions, tracks objects, and highlights detected motion.

4. Vehicle Counting System using Video
A traffic monitoring system must:

A. Read video frames
B. Detect vehicles entering a region of interest (ROI)
C. Track movement across frames
D. Count vehicles and display count

Constraints:

- Avoid double counting
- Handle multiple vehicles simultaneously

Write pseudocode for the complete system, including detection, tracking, counting logic, and visualization using drawing functions.
Answer:
A. Problem Understanding:

- **Input:** Traffic video.
- **Output:** Number of vehicles crossing the counting region.
- **Constraints:** Avoid double counting and handle multiple vehicles.

B. Algorithm Design:

```
START
Open traffic video
Define ROI and counting line
Initialize vehicle tracker and count
WHILE video is available
   Read frame
   Detect vehicles
   Track vehicles
   FOR each vehicle
      Check counting line crossing
      IF vehicle crosses and is not counted
         Increase vehicle count
      END IF
      Draw bounding box
   END FOR
   Display vehicle count
END WHILE
Release video
END
```

C. Use of Control Structures:

- WHILE processes video continuously.
- FOR handles multiple vehicles.
- IF prevents double counting.

D. Pseudocode Structure:
The pseudocode is structured as detection → tracking → line crossing → counting → visualization.
Completeness:
It detects and tracks multiple vehicles, counts them when they cross the line, and avoids duplicate counting.

5. Video Frame Extraction System

A system must:

A. Read video input
B. Extract frames at fixed intervals
C. Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

Write structured pseudocode for frame selection and saving logic.

Answer:

A. Problem Understanding:

- **Input:** Video file.
- **Output:** Selected video frames.
- **Constraints:** Fixed intervals, efficient storage, and no redundant frames.

B. Algorithm Design:

```
START

Open video

Set frame interval

Set frame count = 0

WHILE video is available

  Read frame

  IF frame count MOD interval = 0

    Check for redundant frame

    IF frame is not redundant

      Save frame

    END IF
```

END IF

Increase frame count

END WHILE

Release video

Display saved frame count

END

C. Use of Control Structures:

- WHILE reads video frames continuously.
- IF selects frames at fixed intervals.
- MOD determines the extraction interval.

D. Pseudocode Structure:
The pseudocode follows video reading → interval selection → redundancy checking → frame saving.

Completeness:
The algorithm extracts frames at fixed intervals, avoids redundant frames, saves selected frames, and displays the total saved frames.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs

Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient